In [1]:
```
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\kyoga\Downloads\Salary_data - Salary_data.csv")
df
```

Out[1]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>YearsExperience</th> <th>Salary</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> <th>21</th> <th>22</th> <th>23</th> <th>24</th> <th>25</th> <th>26</th> <th>27</th> <th>28</th> <th>29</th> </tbody> <tbody> </tbody>

1.1	39343
1.3	46205
1.5	37731
2.0	43525
2.2	39891
2.9	56642
3.0	60150
3.2	54445
3.2	64445
3.7	57189
3.9	63218
4.0	55794
4.0	56957
4.1	57081
4.5	61111
4.9	67938
5.1	66029
5.3	83088
5.9	81363
6.0	93940
6.8	91738
7.1	98273
7.9	101302
8.2	113812
8.7	109431
9.0	105582

9.5	116969
9.6	112635
10.3	122391
10.5	121872

In [2]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

In [3]:
```
df.dropna(inplace=True)
```

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

In [5]:
```
df.describe()
```

Out[5]:
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>YearsExperience</th> <th>Salary</th> </thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody></tbody>

30.000000	30.000000
5.313333	76003.000000
2.837888	27414.429785
1.100000	37731.000000
3.200000	56720.750000
4.700000	65237.000000
7.700000	100544.750000
10.500000	122391.000000

In [6]:
```
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
```

In [11]:
```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=42
)
```

In [12]:
```
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
```

Out[12]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /* Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0: #fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2: #ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1: #d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3: cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-default-background: white; --sklearn-color-background: #111; --sklearn-color-border-box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); } .sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path: inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute; width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em; background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /* jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set `[hidden] { display: none !important; }` so we also need the `!important` here to be able to override the default hidden behavior on the sphinx rendered scikit-learn.org. See: https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block !important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */ background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(--sklearn-color-text-on-default-background)); background-size: 2px 100%; background-repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom: 2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex; flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start; width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-items: center; background-color: var(--sklearn-color-background); padding-right: 1em; padding-left: 1em; } /* Toggleable style: style used for estimator/Pipeline/ColumnTransformer box that is clickable and can be expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the default style - Estimators will overwrite some part of the style using the `sk-estimator` class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable { /* Default theme specific background. It is overwritten whether we have a specific estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-background); } /* Toggleable label */ .sk-global label.sk-

toggleable__label { cursor: pointer; display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box; text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */ content: "▸"; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow: visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: "▾"; } /* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /* Colorize estimator box */ .sk-global div.sk-estimator input.sk-toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and "?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-estimator-doc-

link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted, a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-color-fitted-level-3); } /* On hover */ div.sk-estimator:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover, div.sk-label-container:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover, div.sk-label-container:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span, style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display: none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex; margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link:hover span { display: block; } /* "?"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right; font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global a.estimator_doc_link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global a.estimator_doc_link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary { padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; } .estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important; margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even) { background-color: #f6f6f6; } .estimator-table

.parameters-table tr:hover td { background-color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104, 0.232); } /* `table td`is set in notebook with right text-align. We need to overwrite it. */ .estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td { color:rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color:rgb(255, 94, 0); background-color: transparent; } .default td, .estimator-table th { color: black; text-align: left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space: preserve nowrap; } /* Styles for parameter documentation links We need styling for visited so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit; display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to make the entire area of the cell containing the link clickable */ a.param-doc-link::before { position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position: absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area: center right) { .param-doc-description { position-area: center right; position: fixed; margin-left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-paste-icon { background-image: url(background-repeat: no-repeat; background-size: 14px 14px; background-position: 0; display: inline-block; width: 14px; height: 14px; cursor: pointer; border: none; padding: 0; background-color: transparent; } .copy-paste-icon:focus-visible { outline: 1px solid currentColor; outline-offset: 1px; } .features { font-family: monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0); border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom: 0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color: var(--sklearn-color-fitted-level-0); } .features summary { cursor: pointer; display: flex; margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap: 0.5em; padding: .25em; } .features details[open] > summary { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; } .features.fitted details[open] > summary { background-color: var(--sklearn-color-fitted-level-2); border-radius: .20em 0 0 0; } .features details > summary .arrow::before { content: "▸"; color: grey; } .features details[open] > summary .arrow::before { content: "▾"; } .features details:hover > summary { margin: 0; background-color: var(--sklearn-color-unfitted-level-2); } .features.fitted details:hover > summary { margin: 0; background-color: var(--sklearn-color-fitted-level-2); } .features .features-container { max-height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1rem; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted .features-container { background-

color: var(--sklearn-color-fitted-level-0); } .features .image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex; justify-content: center; align-items: center; } .features .copy-paste-icon { background-size: 1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features .features-container table { width: 100%; margin: 0.01em; } .features .features-container table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover { background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; } .features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); } .total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style><body>
LinearRegression()
**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**
<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-1" type="checkbox" checked><label for="sk-estimator-id-1" class="sk-toggleable__label fitted sk-toggleable__label-arrow">
LinearRegression
?Documentation for LinearRegressioniFitted
</label>
<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>

<button type="button" class="copy-paste-icon" aria-label="Copy fit_intercept to clipboard" onclick="copyToClipboard('fit_intercept', this.parentElement.nextElementSibling)" ></button>	fit_intercept fit_intercept: bool, default=Tru Whether to calculate the intercept for this mc set to False, no intercept will be u calculations (i.e. data is expected to be cent
<button type="button" class="copy-paste-icon" aria-label="Copy copy_X to clipboard" onclick="copyToClipboard('copy_X', this.parentElement.nextElementSibling)" ></button>	copy_X copy_X: bool, default=True If True be copied; else, it may be overw
<button type="button" class="copy-paste-icon" aria-label="Copy tol to clipboard" onclick="copyToClipboard('tol', this.parentElement.nextElementSibling)" ></button>	tol tol: float, default=1e-6 The precision solution (`coef_`) is determined b which specifies the convergence criterion underlying solver. `tol` is set as `atol` and `b :func:`scipy.sparse.linalg.lsqr` when fitting on s training data. `tol` is set as of :func:`scipy.linalg.lstsq` when fitting on training data.  .. versionadded:: 1.7 versionchanged:: 1.9  Now supported on dense interpreted as the `cond` para
<button type="button" class="copy-paste-icon" aria-label="Copy n_jobs to	n_jobs n_jobs: int, default=None The num jobs to use for the computation. This wi

clipboard" onclick="copyToClipboard('n_jobs', this.parentElement.nextElementSibling)" ></button>	provide speedup in case of sufficiently problems, that is if firstly `n_targets > secondly `X` is sparse or if `positive` is set to ``None`` means 1 un a :obj:`joblib.parallel_backend` context. ``-1`` r using all processors. See :term:`Glossary <n_ for more d
<button type="button" class="copy-paste-icon" aria-label="Copy positive to clipboard" onclick="copyToClipboard('positive', this.parentElement.nextElementSibling)" ></button>	positive positive: bool, default=False Wh to ``True``, forces the coefficients to be pc This option is only supported for dense array For a comparison between a linear regr model with positive constraints on the regr coefficients and a linear regression withou constraints,<b :ref:`sphx_glr_auto_examples_linear_model_plot_nr .. versionadded

</details>
<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th> <th>Type</th> <th>Value</th> </tbody> <tbody></tbody>

coef_ coef_: array of shape (n_features,) or (n_targets, n_features) Estimated coefficients for the linear regression problem. If multiple targets are passed during the fit (y 2D), this is a 2D array of shape (n_targets, n_features), while if only one target is passed, this is a 1D array of length n_features.	ndarray[float64] (1, 1)	[[9423.82]]
intercept_ intercept_: float or array of shape (n_targets,) Independent term in the linear model. Set to 0.0 if `fit_intercept = False`.	ndarray[float64] (1,)	[25321.58]
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	1
rank_ rank_: int Rank of matrix `X`. Only available when `X` is dense.	int64	np.int64(1)
singular_ singular_: array of shape (min(X, y),) Singular values of `X`. Only available when `X` is dense.	ndarray[float64] (1,)	[14.11]

</details>

<script>/* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-Clause */ function copyToClipboard(text, element) { // Get the parameter prefix from the closest toggleable content const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const fullParamName = paramPrefix ? `${paramPrefix}${text}` : text; const originalStyle = element.style; const computedStyle = window.getComputedStyle(element); const originalWidth = computedStyle.width; const originalHTML = element.innerHTML.replace('Copied!', ''); navigator.clipboard.writeText(fullParamName) .then(() => { element.style.width = originalWidth; element.style.color = 'green'; element.innerHTML = "Copied!";

```
setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false; } document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const parent = element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected copy-paste icon is missing from the DOM structure'); return; } const paramName = element.parentElement.nextElementSibling .textContent.trim().split(' ')[0]; const fullParamName = paramPrefix ? `${paramPrefix}${paramName}` : paramName; element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a `.features` block; its siblings * contain a `details` element and a table containing feature named. * @returns {boolean} Always returns `false` so callers can prevent the default click behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem = element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open; detailsElem.open = true; var content = element.closest('.features').querySelector('tbody') .innerText.trim(); if (!wasOpen) detailsElem.open = false; const rows = content.split('\n').map(row => ` "${row}"`); const formattedText = `[\n${rows.join(',\n')},\n]`; const originalHTML = element.innerHTML.replace('âœ”', ''); const originalStyle = element.style; const copyMark = document.createElement('span'); copyMark.innerHTML = 'âœ”'; copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em'; navigator.clipboard.writeText(formattedText) .then(() => { element.style.display = 'none'; element.parentElement.appendChild(copyMark); setTimeout(() => { copyMark.remove(); element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'orange'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub * https://github.com/skrub-data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_dat
* @returns "light" or "dark" */ function detectTheme(element) { const body = document.querySelector('body'); // Check VSCode theme const themeKindAttr = body.getAttribute('data-vscode-theme-kind'); const themeNameAttr = body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) { const themeKind = themeKindAttr.toLowerCase(); const themeName = themeNameAttr.toLowerCase(); if (themeKind.includes("dark") || themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") || themeName.includes("light")) { return "light"; } } // Check Jupyter theme if (body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if (body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a parent element's color const color = window.getComputedStyle(element.parentNode, null).getPropertyValue('color');
```

const match = color.match(/^rgb\s*\(\s*(\d+)\s*,\s*(\d+)\s*,\s*(\d+)\s*\)\s*$/i); if (match) { const [r, g, b] = [parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3])]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to system preference return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement === null) { console.error(`Element with id ${elementId} not found.`); } else { const theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } } forceTheme('sk-container-id-1');</script></body>

In [14]:
```
model.score(x_train,y_train)

model.score(x_test,y_test)

model.coef_

model.intercept_

import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
model=pickle.load(open('SalaryPred.model','rb'))
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)

print("Estimated Salary for {} years of experience is {}: " .format(yr_of
```

```
Estimated Salary for 44.0 years of experience is [[439969.45722514]]:
```

In [ ]: